

A dark blue vertical bar is on the left. A blue arrow points right from it, containing the date.

7/21/2024

Internship Project Report

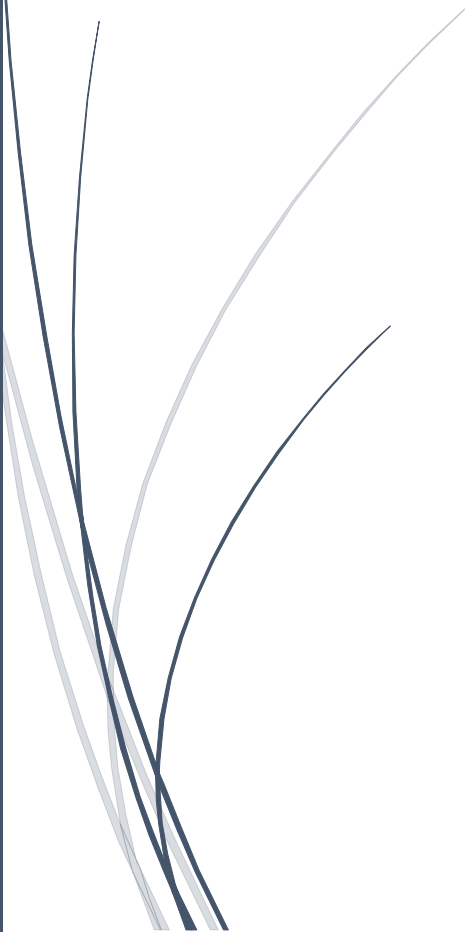
Real-Time Detection and Segmentation
of Cracks

Author

Bhuvnesh Sahu

Mentor

Prasun Agarwal

Several thin, dark blue wavy lines originate from the bottom left and curve upwards and to the right.

Contents

Acknowledgement.....	2
1. Abstract	3
2. Objective	3
3. Solution Overview	3
3.1 Dataset	3
3.2 Models Used	4
3.3 Comparative Analysis	4
4. Methodology	4
4.1 Computer Vision	5
4.2 Object Detection	5
4.3 Segmentation	5
4.4 Object Detection Using YOLOv5	6
4.4 Segment Anything Model (SAM)	7
4.5 Detectron2 for Real-Time Segmentation	9
5. Results	10
5.1 Model Performance	10
5.2 Model Selection	10
6. Application Development	11
6.1 Streamlit App(Link).....	11
6.2 Docker Integration(Link)	11
7. Conclusion.....	11
8. Future Work	11
9. References	11

Acknowledgement

I would like to thank Mr. Sudhir and Mr. Priyavrat Deshpande for providing me the opportunity to pursue this summer internship. My mentors Mr. Prasun Agarwal, have expertly guided and supported me throughout my internship. Their insights and expertise were invaluable in helping me to achieve my goals. Their expert advice and constructive feedback were instrumental in helping me to complete my work. He gave me full freedom to try new things. His understanding nature has a huge part to play in the success of this project.

1. Abstract

This project involves the development of a system for real-time detection and segmentation of cracks in images and videos. Utilizing advanced models such as YOLOv8, Detectron2, and the Segment Anything Model (SAM), this study aims to identify structural issues in 3D-printed building materials. The solution includes a custom-trained dataset and a user-friendly application developed using Streamlit and Docker to ensure accessibility and ease of use.

2. Objective

The primary objective is to create a robust and efficient model capable of detecting and segmenting cracks in real-time using various computer vision techniques and models.

3. Solution Overview

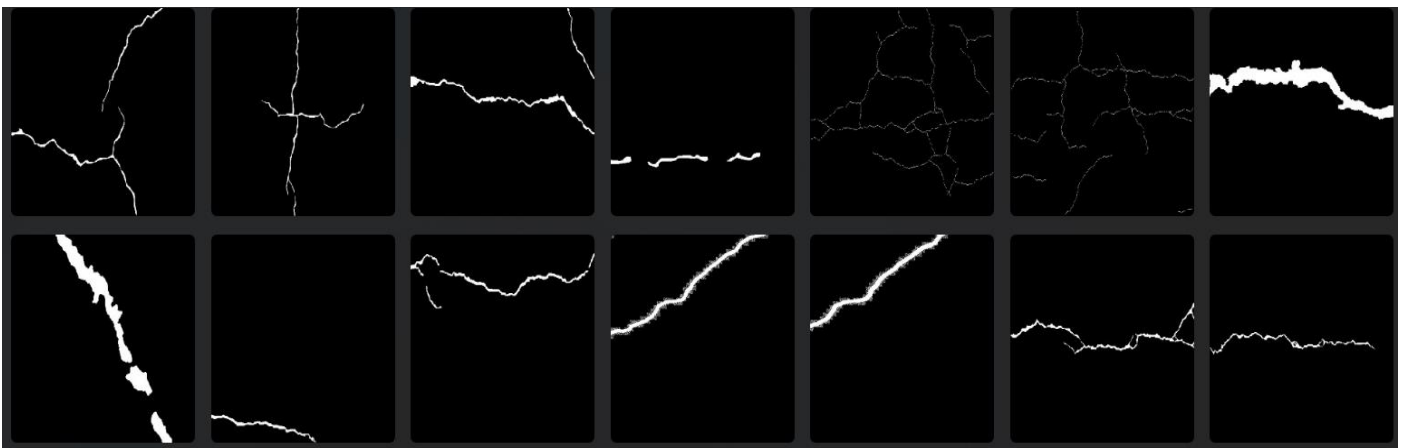
3.1 Dataset

The dataset consists of 430 images (448*448) of various surfaces (walls, roofs, concrete, plaster) with different types of cracks (deep, wide, thin, hairline). The dataset was self-collected and annotated using offline labelling tool LabelMe. For validation of the model I have used 25 images, consisting of all kinds of cracks.

Sample images



Sample Ground Truths



3.2 Models Used

- **YOLOv8**
- **Detectron2**
- **Segment Anything Model (SAM)**

3.3 Comparative Analysis

A comparative analysis will be performed to determine the best model, based on inference time, accuracy, and ease of deployment.

3.3 Resources

To bring Uniformity for the comparative analysis training and inferencing for images and videos is performed on Tesla T4 GPU provided by Google Collab platform.

4. Methodology

4.1 Computer Vision

- Study of how computers can be made to gain high-level understanding from digital images or videos.
- Involves techniques to automate tasks that the human visual system can do.

4.2 Object Detection

- A subfield of computer vision focused on detecting instances of semantic objects of a certain class in digital images and videos.
- Techniques often involve:
 - **Feature extraction:** Identifying important features within an image (e.g., edges, textures, colours).
 - **Machine learning:** Using algorithms like convolutional neural networks (CNNs) to classify and localize objects within an image.
 - **Bounding boxes:** Drawing rectangles around detected objects to specify their location.
 - **Segmentation:** the task includes segmenting the image to precisely outline the objects rather than just placing bounding boxes around them.

4.3 Segmentation

- Divide an image into multiple parts or regions of interest that belong to the same class.
- Explored different segmentation techniques (semantic, instance, and panoptic).
- Focused on binary mask creation for semantic segmentation with a custom dataset.



(a) Image



(b) Semantic segmentation



(c) Instance segmentation

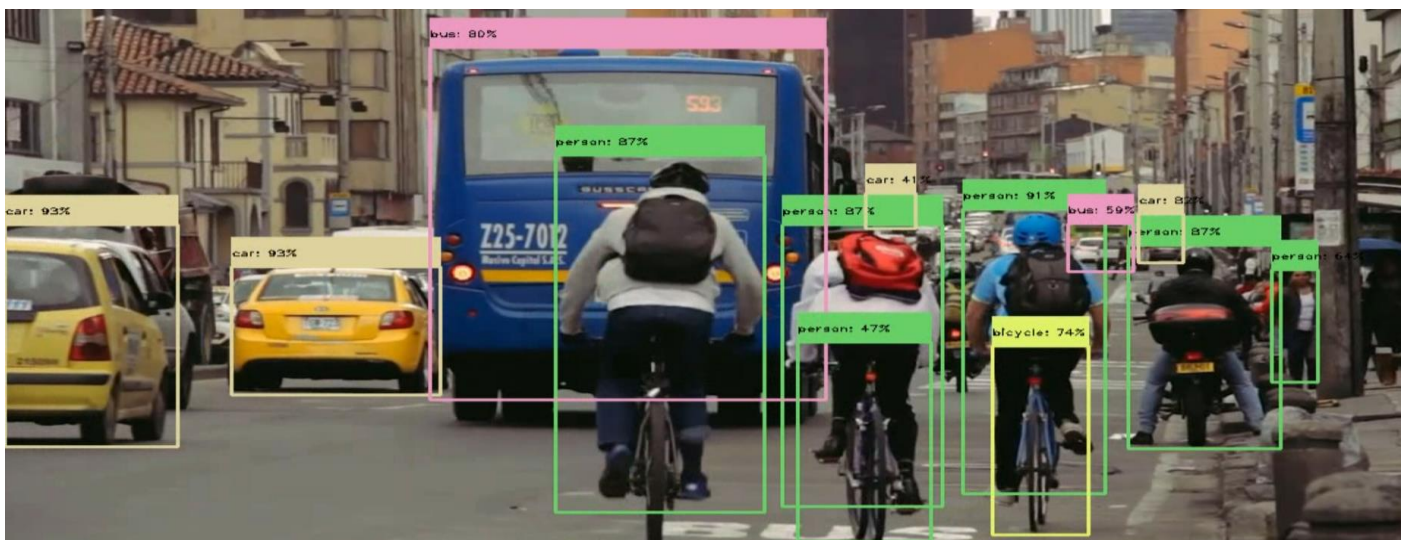


(d) Panoptic segmentation

4.4 Instance Segmentation

- identifies and delineates each object in an image at the pixel level.
- distinguishing between different instances of the same class.
- It combines the elements of object detection and semantic segmentation,
- instance segmentation provides precise boundaries for each object and differentiates between multiple objects within the same category, making it useful for applications such as autonomous driving, medical imaging, and image editing.

- Studied the YOLOv5 framework from Ultralytics' GitHub repository.
- Worked on a sub-project for real-time object detection using the PASCAL VOC dataset.
- Steps involved data preparation, training YOLOv5, and making predictions using OpenCV.



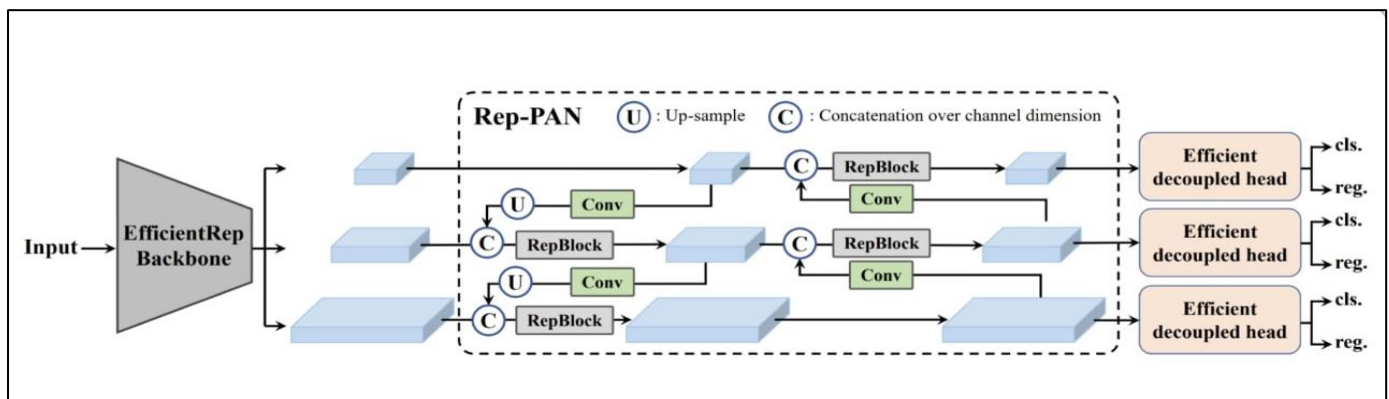
4.3 YOLOv8 for Custom Crack Detection

- Prepared a custom dataset with annotations for training.
- Trained a YOLOv8 model using Ultralytics's library.
- Evaluated the model's performance based on segmentation metrics.

YOLO Training data Sample-

```
0 0.3140877598152425 0.12824956672443674 0.3117782909930716 0.12998266897746968 0.3094688221709007 0.12998266897746968 0.3071593533487298 0.1317157712305026 0.2979214780600462
0.1317157712305026 0.2956120092378753 0.13344887348353554 0.28868360277136257 0.13344887348353554 0.2863741339491917 0.13518197573656845 0.2840646651270208 0.13518197573656845
0.2817551963048499 0.1369150779896014 0.279445727482679 0.1369150779896014 0.26558891454965355 0.14731369150779897 0.26558891454965355 0.14904679376083188 0.2632794457274827
0.15077989601386482 0.2632794457274827 0.16291161178509533 0.26096997690531176 0.16464471403812825 0.26096997690531176 0.1923743500866551 0.2632794457274827 0.19410745233968804
0.2632794457274827 0.19584055459272098 0.26558891454965355 0.1975736568457539 0.2678983833718245 0.1975736568457539 0.2702078521939954 0.19930675909876864 0.2702078521939954
0.2097053726169844 0.27251732101616627 0.21143847487001732 0.27251732101616627 0.225308329289428075 0.2702078521939954 0.2270363951473137 0.2702078521939954 0.2322357019064125
0.27251732101616627 0.2339688041594454 0.27251732101616627 0.23743500866551126 0.2748267898383372 0.2391681109185442 0.2748267898383372 0.24090121317157712 0.27713625866050806
0.24263431542461006 0.27713625866050806 0.24956672443674177 0.2702078521939954 0.25476603119584057 0.2678983833718245 0.25476603119584057 0.2632794457274827 0.2582322357019064
0.26096997690531176 0.2582322357019064 0.2540415704387991 0.2634315424610052 0.2517321016166282 0.2634315424610052 0.2471131639722864 0.2668977469670711 0.24480369515011546
0.2668977469670711 0.2401847575057367 0.2703639514731369 0.23787528868360278 0.2703639514731369 0.23556581986143188 0.2720970537261698 0.23325635103926096 0.2720970537261698
0.23094688221709006 0.2738301559792028 0.22863741339491916 0.2738301559792028 0.22632794457274827 0.2755632582322357 0.22401847575057367 0.2755632582322357 0.22170900692840648
0.2772963604852686 0.21939953810623555 0.2772963604852686 0.21247113163972287 0.2824956672443674 0.21016166281755197 0.2824956672443674 0.21016166281755197 0.28422876949740034
0.28092378752886836 0.29116117850953295 0.28092378752886836 0.292894280762565 0.19630484988452657 0.29636048526863085 0.19630484988452657 0.29809358752166376
0.18937644341801385 0.30329289428076256 0.18937644341801385 0.3050259965337955 0.18475750577367206 0.30849220103986136 0.18475750577367206 0.31022530329289427
0.18244803695150116 0.3119584055459272 0.18244803695150116 0.31369150779896016 0.18013856812933027 0.31542461005199307 0.18013856812933027 0.317157712305026 0.17782909930715934
0.3188908145580589 0.17782909930715934 0.3240901213171577 0.17551963048498845 0.32582322357019067 0.17551963048498845 0.3327556325823224 0.17321016166281755 0.3344887348353553
0.17321016166281755 0.36568457538994803 0.17090069284064566 0.36741767764298094 0.17090069284064566 0.3847487001733102 0.16859122401847576 0.38648100242634317
0.16859122401847576 0.3968804159445407 0.16628175519630484 0.3986135181975737 0.16628175519630484 0.4090121317157712 0.16397228637413394 0.41074523396880414 0.16397228637413394
0.42634315424610053 0.1666281755196305 0.42807625649913345 0.16166281755196305 0.435080866551126516 0.4090121317157712 0.43674176776429807 0.15935334872979215
0.45753899480069327 0.15704387990762125 0.4592720970537262 0.15704387990762125 0.5285961871750433 0.15935334872979215 0.5303292894280762 0.15935334872979215 0.542610051993067
0.15704387990762125 0.5441941074523396 0.15473441108545036 0.5441941074523396 0.15242494226327943 0.5459272097053726 0.14549653579676675 0.5459272097053726 0.14318706697459585
0.5476603119584056 0.14087759815242495 0.5476603119584056 0.14087759815242495 0.5545927209705372 0.14549653579676675 0.5580589254766031 0.14549653579676675 0.5597920277296361
0.14780600461893764 0.561525129982669 0.15011547344110854 0.561525129982669 0.15704387990762125 0.5667244367417678 0.15704387990762125 0.5684575389948007 0.17321016166281755
0.5805892547660312 0.17321016166281755 0.5823223570190641 0.17551963048498845 0.584055459272097 0.17551963048498845 0.58578856152513 0.17782909930715934 0.587521663778163
```

Architecture-



Result Sample-



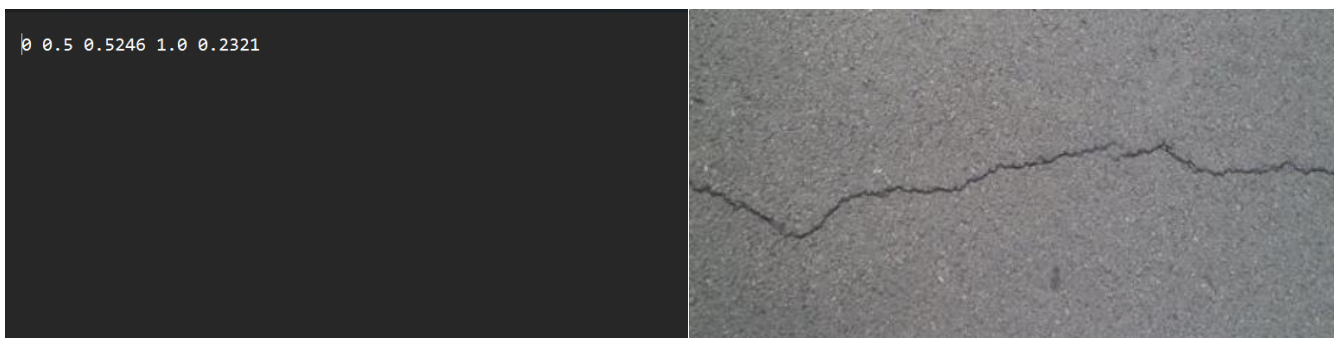
Findings-

- The YOLOv8 model is able to provide inferences with 0.42349 Precision and 0.333 Recall
- The inference time for one image is 9.5 ms

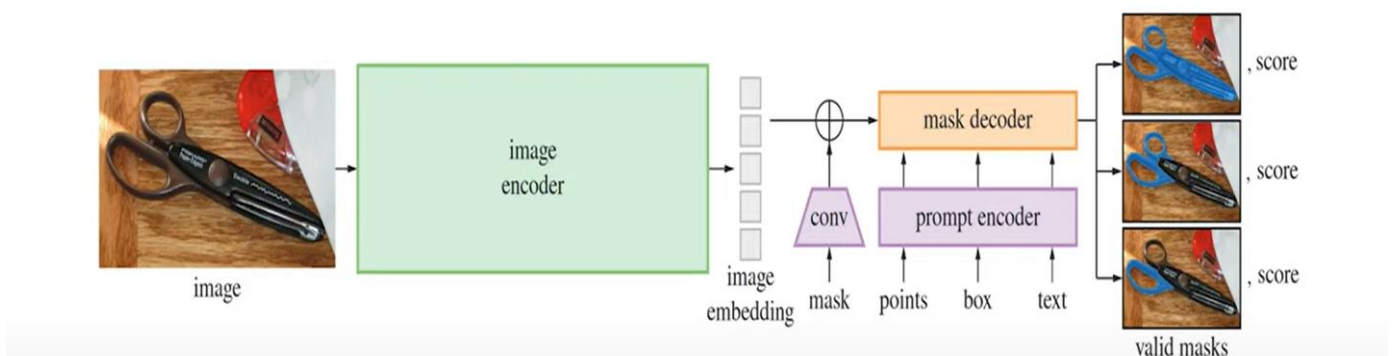
4.4 Segment Anything Model (SAM)

- Implemented SAM for image segmentation.
- SAM uses a Vision Transformer (ViT) for encoding images and generates masks based on prompts (points, boxes, text).
- SAM works by first encoding the image into a high-dimensional vector representation. The image encoder is a vision transformer model, that has been pre-trained on a massive dataset of images.
- The prompt encoder converts the input prompt into a separate vector representation.
- The two vector representations are then combined and passed to a mask decoder, The mask decoder is a lightweight transformer model that predicts the object mask from the image and prompt embeddings

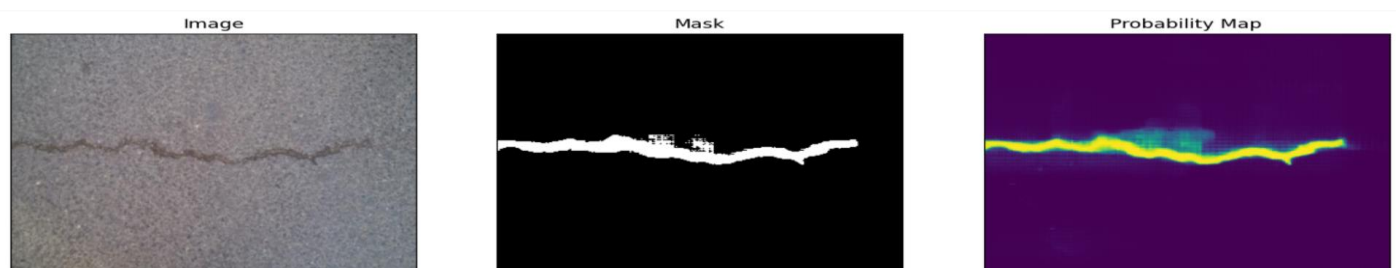
SAM Training Data Sample-



Architecture-



Result Sample-



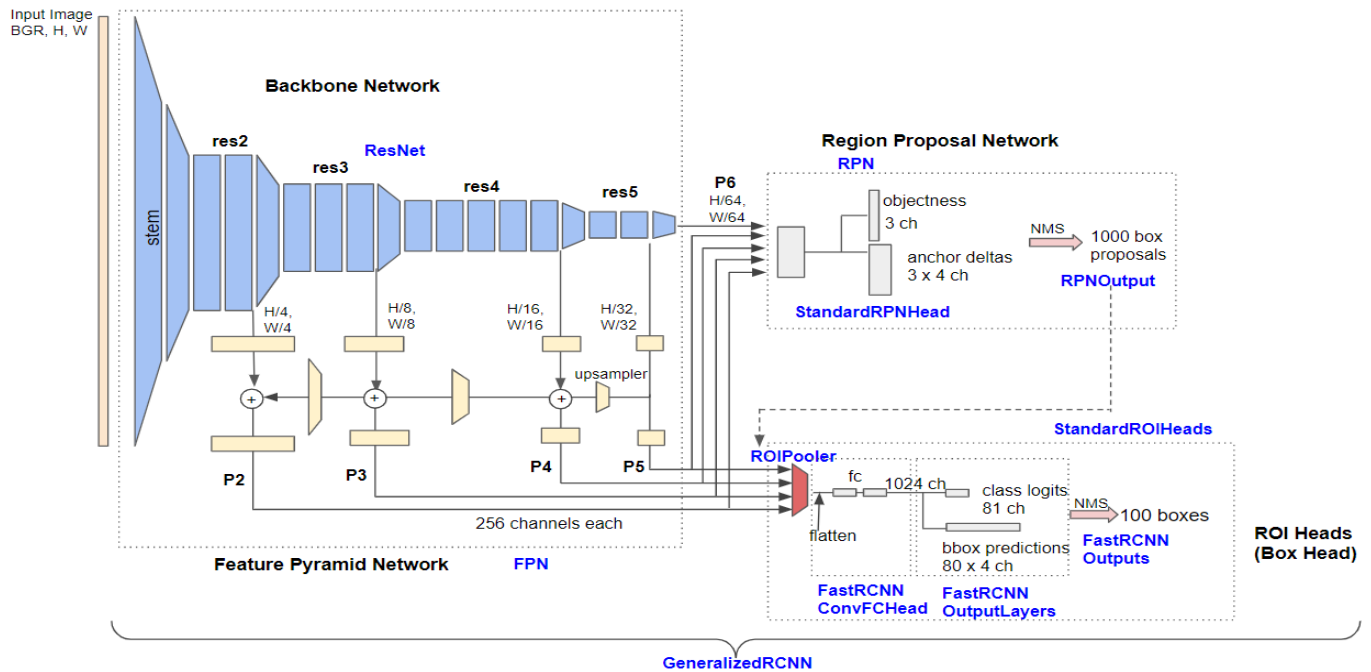
Findings-

- The SAM model is able to provide inferences with 0.43 Precision and 0.56 Recall
- The inference time for one image is 230 ms

4.5 Detectron2 Custom Crack Detection

- Used Detectron2 for object detection and segmentation.
- Trained using the COCO-InstanceSegmentation/mask_rcnn_R_50_FPN_3x.yaml model.
- Converted annotations from YOLO format to COCO format.

Architecture-



Sample Result-



Findings

- The Detectron2 model is able to provide inferences with 0.78 Precision and 0.65 Recall
- The inference time for one image is 40 ms

5. Results

5.1 Model Performance

A comparative table of model performances:

Metrices	SAM	Detectron2	YOLOv8
Precision	0.43	0.78	0.82
Recall	0.56	0.65	0.48
Total Loss	N/A	0.86	1.06
Box Loss	N/A	0.684	0.757
Segmentation loss	1.98	0.958	1.43
FPS	1	7	111
Inference time (MS)	230	40	9.5
No. of Trainable Parameters	91	44.1	3.2
Training time (Iter/Sec)	1.71(2)	4 (6)	3.5 (8)
Model size(MB)	357	334	6.5

- As the SAM model doesn't provide Bounding boxes we can't get the box loss.

5.2 Model Selection

- **YOLOv8**: Best for real-time processing due to its high FPS and low inference time.
- **Detectron2**: Provides better segmentation results with higher precision and recall.
- **SAM**: Most suitable for ambiguous segmentation tasks and complex prompts.

6. Application Development

6.1 Streamlit App([Link](#))

A Streamlit app was developed to provide a user-friendly interface for real-time crack detection and segmentation. The app allows users to upload images or videos and view the results directly.

6.2 Docker Integration([Link](#))

To ensure the application can be easily run on any machine, Docker was used to containerize the Streamlit app and all its dependencies. The Docker image size is approximately **9 GB**, which includes the necessary models and libraries for the application.

Commands to run this image on your local machine-

```
docker run -p 8080:8080 bhuvneshsahu/crack-app
```

7. Conclusion

Based on the comparative analysis, YOLOv8 is recommended for real-time deployment due to its superior processing speed. However, Detectron2 offers better segmentation accuracy and should be considered where precision is critical. SAM can be utilized for tasks requiring flexible and complex segmentation prompts.

8. Future Work

- Enhance the dataset with more diverse crack types and surfaces.
- Improve model training with more epochs and data augmentation.
- Integrate the chosen model into a user-friendly application for real-time monitoring and alerting.

9. References

- YOLOv5 GitHub repository: [Ultralytics YOLO](#)
- YOLOv8 GitHub repository: [YOLOv8](#)
- Detectron2 GitHub repository: [Detectron2](#)
- Segment Anything Model: [SAM](#)